

مشروع المنطق الضبابي والخوارزميات الجينية لتخطيط الرحلات

إعداد الطلاب:

أنس الخطيب – دانيال زين – ريم رشيد الشعرائي – رافت زين الدين – يزن منذر

الجزء الأول: النظام الخبير الضبابي

Section 1 – Fuzzy Expert System

نظام توصية الأماكن السياحية باستخدام المنطق الضبابي

Fuzzy POI Recommendation System for Trip Planning

١. المقدمة

الجزء الأول من المشروع هو بناء Fuzzy Expert System لترشيح الأماكن السياحية والمطاعم أثناء تخطيط الرحلة. حيث تم توثيق ما تم معالجته عن طريقة المنطق الضبابي، وهو مشروع مضمن في مشروع تخرجنا TripSage AI المساعد الذكي في تخطيط الرحلات السياحية. وسوف يستخدم لترشيح الأماكن الأنسب للمستخدم.

٢. توصيف المشكلة

اختيار مكان مناسب للزيارة لا يعتمد عادةً على قرارات حادة مثل نعم أو لا. السائح يستخدم عبارات مرنة مثل: قريب، تقييمه مرتفع، يناسب ميزانيتي، ومتاح في هذا الوقت. هذه المفاهيم متدرجة، لذلك يعد المنطق الضبابي مناسب جداً لأنه يحول هذه العبارات إلى درجات رقمية واضحة بين 0 – 100.

يقيم النظام كل مكان POI سواء كان وجهة سياحية أو مطعم ثم ينتج قيمتين يمكن استخدامها لترتيب الأماكن:

Recommendation Score – Visit Priority

٣. نظرة عامة على النظام

الفكرة العامة بسيطة: يختار المستخدم المدينة وتفضيلاته، ثم يقرأ النظام البيانات من ملفات JSON، ويستخرج منها الـ Features. وبعد ذلك يطبق Fuzzy Expert System لانتاج ترتيب أماكن.



٤. البيانات والتوليد التلقائي للمدخلات

يقرأ النظام ملفي attraction.json و restaurants.json لكل مدينة. من مسارات يحددها المستخدم في أعلى الـ notebook.

بعد ذلك يوحد الحقول في جدول واحد، ويصفي الأماكن حسب اسم المدينة، ثم يحسب المدخلات الستة تلقائياً.

- **Distance Score**: يحسب من مسافة Haversine بين المكان وموقع الفندق-المستخدم، أو مركز المدينة المقدر إذا لم يوجد موقع محدد.
- **Rating Score**: يحول الـ rating من مجال 5-0 إلى مجال 100-0، ومع القيم المفقودة يستخدم قيمة وسطية.
- **Popularity Score**: يستخدم log normalization على الـ num_reviews حتى لا تسيطر الأماكن المشهورة جداً على النتيجة.
- **Budget Match**: يقارن ميزانية المستخدم مع الـ price_level أو price أو fee أو أقل سعر متاح.
- **Interest Match**: يستخدم keyword matching بين اهتمامات المستخدم وبين name – description – category – subcategory – subtype – cuisine – dietary_restrictions – establishments_types – location_subtype – ranking_category – ranking_subcategory
- **Availability Score**: يفحص الـ hours.week_ranges للوقت المختار، ويقيم بثلاث قيم: 100 إذا كان مفتوحاً – 60 إذا كانت الساعات غير معروفة، و 20 إذا كان مغلقاً.

تم إرفاق ملفات الـ JSON لتوضيح شكل البيانات المستخدمة.

٥. المتغيرات المدخلة والمجموعات الضبابية (Input Variables)

المتغير والمدى	المعنى	المجموعات الضبابية
Distance Score 0–100	درجة قرب المكان من موقع الفندق أو المستخدم	Poor / Acceptable / Excellent
Rating Score 0–100	التقييم بعد تحويله من 5–0 إلى 100–0	Low / Good / Excellent
Popularity Score 0–100	لعدد المراجعات log normalization شعبية المكان بعد	Unpopular / Known / Popular
Budget Match 0–100	مدى توافق سعر المكان مع مستوى ميزانية المستخدم	Poor / Acceptable / Ideal
Interest Match 0–100	مدى تطابق التصنيف أو المطبخ أو الوصف مع اهتمامات المستخدم	Weak / Moderate / Strong
Availability Score 0–100	مدى ملائمة ساعات العمل لليوم والوقت المختارين	Closed/Unknown / Maybe Available / Available

٦. المتغيرات الناتجة والمجموعات الضبابية (Output Variables)

المتغير والمدى	المعنى	المجموعات الضبابية
Recommendation Score 0–100	جودة التوصية النهائية للمكان	Low / Medium / High
Visit Priority 0–100	أولوية إدراج المكان ضمن خطة الزيارة	Low / Medium / High

٧. دوال العضوية (Membership Functions)

يستخدم النظام دوال الـ Triangular و Trapezoidal لأنها واضحة وسهلة الضبط والشرح. كل متغير يعمل على Universe من 0-100، وتسمح مناطق التداخل بأن تنتمي القيمة إلى أكثر من Fuzzy Set بدرجات مختلفة.

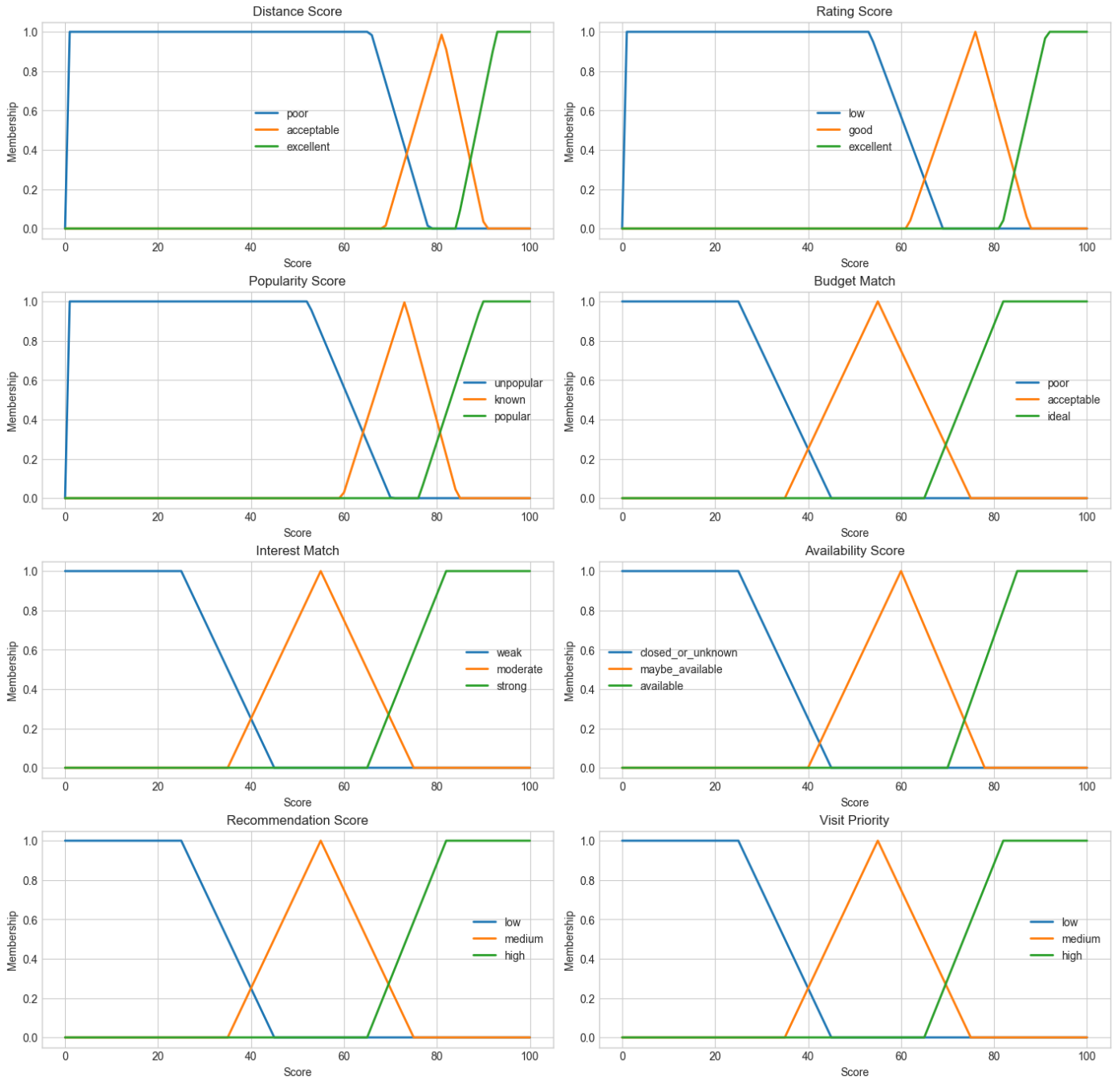
- **Low / Poor**: غالباً Trapezoidal Membership Function لتمثيل المجال المنخفض.
- **Medium / Acceptable**: غالباً Triangular Membership Function بقيمة في منتصف المجال.
- **High / Excellent**: غالباً Trapezoidal Membership Function لتمثيل المجال المرتفع قرب الـ 100.

اعتمد النظام على فكرة Auto-calibrated Fuzzy Logic، حيث لا يتم تثبيت حدود Membership Functions بشكل عشوائي، بل يتم ضبطها اعتماداً على خصائص البيانات نفسها مثل الشرائح المنوية (quantiles) والتجميع (Clustering) والقيم الافتراضية الاحتياطية الآمنة (safe fallback defaults). على سبيل المثال، يتم تحليل قيم مثل rating_score و distance_score و popularity_score لاختيار حدود مناسبة لفئات مثل Low و Medium و High. هذا يجعل النظام أكثر مرونة، لأن شكل الـ Membership Functions يتغير حسب المدينة والبيانات المتاحة بدل استخدام نفس القيم الثابتة دائماً. بهذه الطريقة تصبح التوصيات أقرب للواقع وأكثر تكيفاً مع اختلاف جودة وكثافة البيانات بين مدينة وأخرى.

تم رسم الـ Membership Functions باستخدام مكتبة matplotlib وهذا يساعد على فهم طريقة تحويل القيم الرقمية إلى مفاهيم لغوية.

Fuzzy Logic | Section 1: Fuzzy Expert System

بحيث كان شكل ال Membership functions للمدخلات والمخرجات في مدينة Las Vegas كالتالي:



٨. قاعدة المعرفة والقواعد

تمثل الـ Rule Base المعرفة الخبيرة داخل النظام. يحتوي النظام على 14 قاعدة. يمكن تفعيل أكثر من قاعدة في الوقت نفسه، ثم تدمج النتائج قبل الـ Defuzzification.

#	الشروط If	النتيجة Then
1	Interest Match = Strong AND Rating Score = Excellent AND Distance Score = Excellent	Recommendation Score = High Visit Priority = High
2	Interest Match = Strong AND Budget Match = Ideal AND Availability Score = Available	Recommendation Score = High Visit Priority = High
3	Rating Score = Excellent AND Popularity Score = Popular AND Interest Match = Moderate	Recommendation Score = High Visit Priority = Medium
4	Distance Score = Poor AND Budget Match = Poor	Recommendation Score = Low Visit Priority = Low
5	Interest Match = Weak	Recommendation Score = Low Visit Priority = Low
6	Availability Score = Closed/Unknown AND Interest Match is not Strong	Recommendation Score = Low Visit Priority = Low
7	Budget Match = Acceptable AND Rating Score = Good AND Interest Match = Moderate	Recommendation Score = Medium Visit Priority = Medium
8	Distance Score = Excellent AND Rating Score = Good	Recommendation Score = Medium Visit Priority = Medium
9	Popularity Score = Unpopular AND Rating Score = Low	Recommendation Score = Low Visit Priority = Low
10	Interest Match = Strong AND Distance Score = Poor AND Rating Score = Excellent	Recommendation Score = Medium Visit Priority = Medium
11	Availability Score = Available AND Budget Match = Ideal AND Rating Score = Excellent	Recommendation Score = High Visit Priority = High
12	Most signals are Medium or Acceptable	Recommendation Score = Medium Visit Priority = Medium
13	Interest Match = Strong AND Popularity Score = Popular AND Availability Score = Available	Recommendation Score = High Visit Priority = High
14	Budget Match = Ideal AND Distance Score = Excellent AND Interest Match = Moderate	Recommendation Score = High Visit Priority = Medium

٩. محرك الاستنتاج

يستخدم النظام Mamdani Inference، وهي طريقة شائعة وسهلة الفهم في Fuzzy Expert Systems. تبدأ العملية بتحويل القيم الرقمية إلى درجات عضوية، ثم تقييم شروط القواعد باستخدام عمليات AND و OR، وبعد ذلك يتم تجميع مخرجات القواعد الفعالة لتكوين Fuzzy Output لكل Output Variable.

١٠. تحويل المخرجات إلى قيمة رقمية

بعد تجميع مخرجات القواعد نحتاج إلى قيمة رقمية قابلة للترتيب. يستخدم المشروع Centroid Method لحساب مركز مساحة Membership Function الناتجة. هذه الطريقة تعطي نتيجة متوازنة بين 0 و 100، ولذلك تناسب Recommendation Score و Visit Priority.

١١. واجهة المستخدم

توفر واجهة Jupyter Notebook المبنية باستخدام ipywidgets طريقة مباشرة لتعديل الإعدادات وإعادة تشغيل التوصية دون تعديل الخلايا البرمجية في كل مرة. يمكن للمستخدم تحديد:

- اسم المدينة (City Name)
- مستوى الميزانية (Budget Level)
- اهتمامات المستخدم (User Interests)
- اليوم والوقت المفضل للزيارة (Preferred Day / Preferred Visit Time)
- إحداثيات الفندق بشكل اختياري (Hotel Latitude / Hotel Longitude)
- عدد النتائج المعروضة (Top N)

City	Las Vegas	Budget	medium	Day	Tuesday
Interests	Nature & Parks, Sights & Landmarks, Museums, American, Italian				
Time			840	Top N	
Hotel lat	blank = estimate center		Hotel lon	blank = estimate center	

تعرض الواجهة جدول الترتيب، Bar Chart لأفضل النتائج، وشرحاً مختصراً لأفضل POI.

١٢. الاختبار والتحقق

لا يعتبر هذا التقييم تدريباً لنموذج Machine Learning، بل هو فحص منطقي للتأكد من أن Rule Base تتصرف بالاتجاه المتوقع. تتم المقارنة بين expected label و predicted label باستخدام التصنيف التالي:

Label	المجال
Low	من 0 إلى 39
Medium	من 40 إلى 69
High	من 70 إلى 100

في التشغيل الحالي تم تحميل 30 attraction و 30 restaurant، وبعد التصنيفية ظهر 59 POI لمدينة Las Vegas. وصلت Synthetic Validation Accuracy إلى 100%، بينما كانت Manual Validation Accuracy حوالي 33.3%. هذه النتيجة لا تعني وجود خطأ في Mamdani Inference، لكنها تشير إلى أن labels اليدوية أو قواعد التوقع تحتاج إلى معايرة أفضل لتتوافق مع Rule Base.

١٣. المتطلبات وطريقة التشغيل

لتشغيل المشروع من Jupyter Notebook، نثبت المكتبات التالية ثم تشغل الخلايا من الأعلى إلى الأسفل:

```
pip install numpy pandas matplotlib scikit-fuzzy ipywidgets
```

- تعديل مسارات RESTAURANTS_PATH و ATTRACTIONS_PATH في أعلى notebook.
- تحديد CITY_NAME والميزانية والاهتمامات واليوم والوقت المناسب للزيارة.
- تشغيل الخلايا بالترتيب ومراجعة Membership Functions والجدول النهائي والرسوم ونتائج Validation.

١٤. القيود والتطوير المستقبلي

النظام مناسب كنسخة أكاديمية واضحة، لكنه ما زال يعتمد على جودة البيانات المحلية. أهم القيود الحالية هي:

- Interest Match يعتمد على keyword matching بسيط.
- بعض الأماكن قد لا تحتوي على rating أو hours أو price.
- النظام لا ينفذ route optimization ولا يبني itinerary كامل.
- النتائج تحتاج إلى معايرة إضافية إذا تغيرت المدينة أو نوع البيانات.

يمكن تطويره لاحقاً باستخدام NLP لمطابقة الاهتمامات بشكل أذكى، وإضافة real-time APIs، وتحسين Rule Base، وربط النتائج مع itinerary optimization.

تحسين برنامج الرحلة متعدد الأيام باستخدام Itinerary Optimization

١. المقدمة

يوثق هذا الجزء نظام Genetic Algorithm المستخدم بعد أن ينشئ LLM Planner Agent برنامج الرحلة الأولي. يحتوي البرنامج الأولي على مجموعة من نقاط الاهتمام POIs موزعة على عدة أيام، ثم تستقبل الخوارزمية هذه الخطة وتحاول تحسين ترتيب الزيارات وتوزيعها اليومي. لا تقوم الخوارزمية بكتابة أوصاف الأماكن ولا تختار المحتوى النصي للمستخدم. مهمتها هي Optimization فقط: تقليل كلفة التنقل، احترام ميزانية الوقت اليومية قدر الإمكان، مراعاة ساعات العمل عند توفرها، والحفاظ على استخدام متوازن للأيام.

٢. توصيف المشكلة

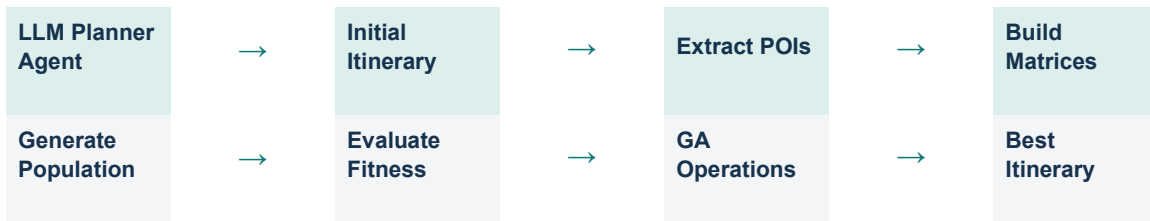
قد تكون الخطة الناتجة من LLM منطقية من ناحية اختيار الأماكن، لكنها ليست بالضرورة الأفضل كمسار. فقد تظهر أماكن متباعدة في اليوم نفسه، أو يكون أحد الأيام مزدحماً والآخر خفيفاً، أو يتجاوز المسار الوقت المتاح. لذلك نستخدم البحث التطوري للوصول إلى ترتيب أفضل بين عدد كبير من البدائل الممكنة.

الهدف هو إيجاد برنامج رحلة متعدد الأيام يقلل العناصر التالية:

- إجمالي زمن التنقل والزيارة total travel time
- إجمالي مسافة المسار total travel distance
- تجاوز ميزانية الوقت اليومية ومخالفات ساعات العمل
- عدم التوازن بين الأيام والأيام الفارغة أو قليلة الاستفادة

٣. نظرة عامة على النظام

يبدأ التدفق من الخطة الأولية، ثم تفرد قائمة الأماكن، وتبنى مصفوفتا الزمن والمسافة مرة واحدة. بعد ذلك ينشأ Population متنوع، وتقيم الحلول باستخدام Fitness Function، ثم تتكرر العمليات الجينية حتى الوصول إلى أفضل Chromosome.



٤. تمثيل الكروموسوم (Chromosome Representation)

التمثيل في الكود هو تمثيل مبني على المسارات. route-based representation. يحتوي كل Chromosome على قائمة routes ، وكل مسار يمثل يوماً واحداً من الرحلة، بينما تمثل الأرقام فهارس الأماكن داخل القائمة الكلية بعد تسطيح البرنامج الأولي.

اليوم	المسار داخل الكروموسوم	المعنى
Day 1	[0, 3, 5]	زيارة الأماكن ذات الفهارس 0 ثم 3 ثم 5
Day 2	[1, 4]	زيارة المكانين 1 ثم 4
Day 3	[2, 6, 7]	زيارة الأماكن 2 ثم 6 ثم 7

الفندق لا يخزن كعنصر POI داخل الكروموسوم. لكنه يمثل العقدة رقم 0 في مصفوفتي الزمن والمسافة، ويستخدم كنقطة بداية ونهاية لكل يوم، بينما يتحول فهرس المكان i إلى العقدة $i + 1$ أثناء التقييم.

يجب أن يظهر كل POI مرة واحدة فقط. ولهذا تطبق دالة Repair لإزالة التكرار والقيم غير الصحيحة، وإضافة الأماكن المفقودة إلى أقصر الأيام، والتأكد من أن عدد المسارات يساوي عدد أيام الرحلة.

٥. إنشاء المجتمع الابتدائي (Initial Population)

لا يعتمد البحث على ترتيب LLM وحده. يبدأ النظام بنسخة تحافظ على البرنامج الأصلي، ثم يضيف مرشحين عشوائيين بطرق مختلفة حتى يحصل على مجتمع متنوع يمكنه استكشاف مساحة حلول أكبر.

1. كروموسوم يحافظ على ترتيب وتوزيع برنامج LLM الأصلي.
 2. كروموسومات عشوائية متوازنة توزع الأماكن بالتقريب على الأيام.
 3. كروموسومات عشوائية تحافظ على العدد الأصلي للأماكن في كل يوم.
 4. كروموسومات عشوائية بالكامل لزيادة التنوع ومنع البحث من الانحصار في ترتيب واحد.
- في التنفيذ الحالي تشكل الحلول المتوازنة تقريباً أول 35% من المجتمع، ثم حلول التوزيع الأصلي حتى 70%، والباقي حلول عشوائية بالكامل. ويمكن أيضاً توسيع الفكرة باستخدام بيانات POI المتاحة لتوليد مسارات أولية أكثر تنوعاً قبل بدء الحلقة التطورية.

٦. دالة الملاءمة (Fitness Function)

التنفيذ يقلل قيمة Cost، ثم يحولها إلى Fitness لاستخدامها في الاختيار. العلاقة المستخدمة هي:

$$\text{Fitness} = 1 / (1 + \text{Cost})$$

كلما انخفضت Cost أصبح الحل أفضل، وكلما ارتفعت Fitness زادت فرصة اختيار الكروموسوم كأب. تتكون الكلفة من مجموع موزون للمقاييس التالية:

Metric	المعنى	الوزن الافتراضي
travel_time	زمن المسار الكلي، ويشمل التنقل ومدة الزيارة	1.0
travel_distance	المسافة الكلية بالمتري، وتظهر في النتائج بالكيلومتر	0.001
over_budget	عقوبة لكل دقيقة تتجاوز ميزانية اليوم	10,000
opening_hours	عقوبة لكل زيارة خارج ساعات العمل المتاحة	5,000
balance	عقوبة الانحراف المعياري لأزمة الأيام	2.0
empty_day	عقوبة إذا بقي يوم من دون أماكن	20,000

تسمح الأوزان بتحديد ما يهتم به المحسن أكثر. فرفع وزن تجاوز الوقت يجعل الحلول غير الممكنة مكلفة جداً، بينما رفع وزن التوازن يعطي أولوية أكبر لتقارب أحمال الأيام.

٧. العمليات الجينية (Genetic Operators)

(a) الاختيار Selection:

يستخدم النظام Tournament Selection: تختار مجموعة صغيرة عشوائياً من المجتمع، ثم يفوز الكروموسوم ذو أعلى Fitness ويصبح أحد الوالدين.

(b) العبور Crossover:

يفرد النظام مسارات الوالدين إلى ترتيبين كاملين للأماكن، ويطبق Ordered Crossover - OX. ينسخ جزءاً من الوالد الأول، ثم يملأ المواقع الباقية وفق ترتيب الوالد الثاني، وبعدها يقسم الناتج مرة أخرى إلى مسارات يومية باستخدام أطوال مأخوذة من الوالد

(c) الطفرة Mutation:

- تبديل مكانين داخل اليوم نفسه.
- نقل مكان من يوم إلى يوم آخر.
- عكس مقطع داخل مسار يوم واحد.
- تبديل مكانين بين يومين مختلفين.

(d) الإصلاح Repair:

بعد العبور أو الطفرة، يصلح الكروموسوم للتأكد من عدم فقدان أي مكان أو تكراره، وإزالة الفهارس غير الصالحة، والمحافظة على عدد أيام الرحلة.

(e) النخبوية Elitism:

تنسخ أفضل الكروموسومات مباشرة إلى الجيل التالي حتى لا تضيع الحلول الجيدة بسبب العمليات العشوائية.

(f) التوقف المبكر Early Stopping:

يمكن إنهاء البحث إذا لم تحسن أفضل كلفة خلال عدد محدد من الأجيال، مما يقلل وقت التشغيل عندما يصل البحث إلى حالة استقرار.

٨. الحلقة التطورية (Evolutionary Loop)

تنفذ الحلقة الرئيسية في الدالة `run_ga` بالترتيب التالي:

1. إنشاء المجتمع الابتدائي المتنوع.
 2. تقييم كل كروموسوم وحساب الكلفة والملاءمة.
 3. ترتيب المجتمع والاحتفاظ بالحلول النخبوية.
 4. اختيار الوالدين بطريقة البطولة.
 5. تطبيق العبور والطفرة ثم الإصلاح.
 6. تقييم الجيل الجديد وتحديث أفضل حل وتاريخ التطور.
 7. التكرار حتى عدد الأجيال المحدد أو تحقق التوقف المبكر.
 8. إرجاع أفضل برنامج رحلة تم العثور عليه.
- يسجل النظام في كل جيل أفضل ومتوسط `Cost`، وأفضل ومتوسط `Fitness`. ويمكن استخدام هذا السجل لرسم منحنى التقارب ومعرفة متى توقف التحسن.

٩. المخرجات النهائية (Outputs)

تعيد الدالة `solve_multi_day_ga` سلسلة JSON منظمة تحتوي على:

المفتاح	المحتوى
<code>optimization_type / settings</code>	نوع التحسين وإعدادات التشغيل والأوزان ومصادر البيانات
<code>ga_metrics</code>	أفضل كلفة وملاءمة، جيل التقارب، عدد الأجيال، زمن التشغيل، وقابلية الحل
<code>fitness_breakdown</code>	تفصيل الزمن والمسافة والعقوبات والكلفة النهائية
<code>baseline_comparison</code>	مقارنة البرنامج الأصلي مع البرنامج المحسن ونسبة التحسن
<code>optimized_itinerary</code>	الخطة المحسنة مرتبة حسب Day / Morning / Afternoon / Evening
<code>daily_summaries / routes</code>	ملخصات الأيام والمسارات والعودة إلى الفندق
<code>evolution_history</code>	تاريخ الأجيال المستخدم في رسم تقدم الخوارزمية

توضح الملخصات اليومية وقت البداية والنهاية، عدد الأماكن، زمن التنقل والزيارة، المسافة، والزمن الكلي لكل يوم. وهذا يجعل النتيجة قابلة للعرض في الواجهة أو للتحليل لاحقاً.

١٠. واجهة المستخدم أو طريقة التشغيل (User Interface)

يمكن استخدام التنفيذ الحالي مباشرة من Python أو داخل Jupyter Notebook. أهم المدخلات هي إحداثيات الفندق، ميزانية الوقت، ساعة البداية، اليوم الأسبوعي، وإعدادات الخوارزمية وملفات بيانات الأماكن الخام.

Parameter	الوظيفة	مثال
hotel_lat / hotel_lon	إحداثيات نقطة بداية ونهاية المسار	36.112 / -115.173
daily_time_budget_hours	الوقت المتاح لكل يوم	8
start_hour / weekday_index	ساعة البداية ورقم اليوم لفحص ساعات العمل	9 / 0
population_size / generations	حجم المجتمع وعدد الأجيال	80 / 200
crossover_rate / mutation_rate	احتمال العبور والطفرة	0.9 / 0.2
random_seed	تثبيت العشوائية لإعادة التجربة	42
raw_poi_files	ملفات الأماكن لإثراء ساعات العمل ومدد الزيارة	attractions.json, restaurants.json

```
result_json = solve_multi_day_ga(
    itinerary=initial_itinerary,
    hotel_lat=36.112,
    hotel_lon=-115.173,
    raw_poi_files=["attractions.json", "restaurants.json"],
    daily_time_budget_hours=8,
    population_size=80,
    generations=200,
    mutation_rate=0.2,
)
```

١١. الاختبار والتحقق (Testing and Validation)

تتم عملية التحقق بمقارنة برنامج LLM الأصلي مع أفضل حل أنتجته الخوارزمية. تشمل المقارنة الكلفة الكلية والمسافة والزمن وتوازن الأيام، مع فحص وجود حل يحقق قيود الوقت وساعات العمل وعدم وجود أيام فارغة.

- استخدام baseline comparison قبل التحسين وبعده.
- مراجعة GA metrics وجيل التقارب وزمن التشغيل.
- رسم أفضل ومتوسط كلفة عبر الأجيال باستخدام plot_evolution_history.
- رسم أزمنة المسارات اليومية باستخدام plot_daily_route_times.

الجدول التالي يعرض نتائج التشغيل التجريبي المحفوظة في المشروع. تحسنت الكلفة والمسافة والزمن، لكن توازن الأيام أصبح أضعف ولم يتحقق حل ممكن بالكامل، لذلك تحتاج الأوزان أو عدد الأيام أو مجموعة الأماكن إلى معايرة إضافية.

Metric	Original Itinerary	GA Optimized
Total Cost	5,543,712.645	4,804,540.219
Total Distance	305.488 km	218.484 km
Total Time	1,992.633 min	1,918.717 min
Daily Balance	40.597 min	118.170 min
Improvement %	-	13.33%

في هذه false كانت feasible_solution_found وتوقف التشغيل بعد 126 جيلاً بسبب عدم ظهور تحسن جديد. قيمة وصل أفضل حل في الجيل 86، التجربة.

١٢. القيود والتطوير المستقبلي

أهم القيود الحالية هي:

- الخوارزمية احتمالية stochastic ، لذلك قد تتغير النتائج عند تغيير random seed.
- تحتاج قيم حجم المجتمع وعدد الأجيال ومعدل الطفرة وأوزان العقوبات إلى معايرة حسب الرحلة.
- تعتمد الدقة على جودة مصفوفتي الزمن والمسافة وعلى اكتمال ساعات العمل في البيانات الخام.
- قد يكون التنفيذ أبطأ من طرق التحسين الحتمية عندما يصبح عدد الأماكن كبيراً.
- المنهج الحالي يحسن الأماكن المختارة، لكنه لا يستبدل منطق LLM Planner Agent في فهم المستخدم واختيار المحتوى.

يمكن تطوير النظام لاحقاً من خلال:

- اختيار أماكن أفضل من مجموعة مرشحين أكبر قبل بناء الكروموسوم.
- دمج درجات التوصية الضبابية Fuzzy Recommendation Scores داخل الكلفة أو المجتمع الأولي.
- إضافة قيود أدق للمطاعم وأوقات الغداء والعشاء وتفضيلات المستخدم.
- مقارنة النتائج مع OR-Tools وإضافة بحث محلي بعد الطفرة.
- عرض المسارات على خريطة وربط تاريخ التطور بلوحة واجهة بسيطة.

١٣. المتطلبات وطريقة التشغيل

يتطلب التشغيل Python ومكتبتَي numpy و matplotlib، إضافة إلى ملفات المشروع ومزود لمصفوفة الزمن والمسافة مثل Geoapify. يجب أن تكون الملفات التالية متاحة في مجلد optimization/GA:

- ga_optimizer.py لتنفيذ الخوارزمية وبناء المخرجات.
 - ga_utils.py للأدوات المساعدة الخاصة بالكروموسومات والمسارات.
 - raw_poi_utils.py لقراءة البيانات الخام وساعات العمل ومدد الزيارة.
 - utils.py لتسطيح البرنامج والاتصال بمصفوفة المسافات.
- pip install numpy matplotlib